



UNIVERSIDAD AUTÓNOMA
DE BUCARAMANGA

FACULTAD DE INGENIERÍA

TALLER N°1

PREPROCESAMIENTO DE DATOS Y
CLASIFICACIÓN CON REDES NEURONALES

ANÁLISIS DE LA ARQUITECTURA Y LA TASA DE
APRENDIZAJE EN SCIENXLAB PLAYGROUND

UNIDAD: PREPROCESAMIENTO Y REDES NEURONALES

PRESENTA:

José Alejandro Téllez Prada – U00060479

PROGRAMA

Gestión de Sistemas de Información

Colombia, Bucaramanga, Santander

17/08/2026

Índice

1. Objetivo	3
2. Paso 1 – Preprocesamiento de Datos	3
3. Paso 2 – Archivo JSON Generado	3
4. Paso 3 – Análisis de Experimentos	3
4.1. Experimento 1: Tasa de aprendizaje (arquitectura fija [2, 4, 1])	3
4.2. Experimento 2: Arquitectura de la red ($lr = 0.03$)	4
5. Mejor Solución y Justificación Teórica	4
6. Evidencia Visual (ScienxLab)	5
6.1. Red Entrenada Final ($lr = 0.003$)	5
6.2. Frontera de Decisión	6
6.3. Comparación de Tasa de Aprendizaje ($lr = 0.3$ con oscilaciones)	7
7. Código de la Red (Pesos y Sesgos)	7
7.1. Pesos y sesgos calculados en Python / Google Colab	7
7.2. Parámetros Registrados en ScienxLab	8
8. Deployment con Streamlit	8
8.1. Descripción de la aplicación	8
8.2. Instalación y ejecución	9
8.3. Tecnologías utilizadas	9
8.4. Evidencia de la aplicación	10

1. Objetivo

Preparar y preprocesar el conjunto de datos real `pacientes.csv` (302 pacientes) para transformarlo al formato JSON requerido por ScienxLab Playground, e implementar un modelo de Red Neuronal Artificial para clasificación binaria de problemas cardíacos, analizando el impacto de la arquitectura de la red y de la tasa de aprendizaje (*learning rate*).

2. Paso 1 – Preprocesamiento de Datos

Etapas	Detalle	Resultado
Descarga	CSV desde GitHub (adiaccla/bigdata)	302 filas, 3 columnas:
Limpieza	Eliminación de filas con nulos	302 → 298 filas
Selección/renombrado	edad → x, colesterol → y, problema_cardiaco → label	2 variables predictoras
Mapeo target	Clase positiva (1) → 1 , clase negativa (0) → -1	147 positivos / 151 negativos
Estandarización	Z-score: $z = (v - \mu)/\sigma$, multiplicado por 2	medias ≈ 0 , desviaciones ≈ 1

3. Paso 2 – Archivo JSON Generado

Archivo: `pacientes_preprocesado.json` (298 registros), compatible con ScienxLab Playground:

```
[
  {"x": 1.914233, "y": -0.538876, "label": 1},
  {"x": -3.809995, "y": 0.118942, "label": -1},
  {"x": -2.929345, "y": -1.661037, "label": -1}
]
```

4. Paso 3 – Análisis de Experimentos

Condiciones fijas en todos los experimentos: Classification, Squared error, ReLU (ocultas), sin regularización, 70 % entrenamiento / 30 % prueba (208/90 muestras).

4.1. Experimento 1: Tasa de aprendizaje (arquitectura fija [2, 4, 1])

learning rate	Pérdida train	Pérdida test	Acc train	Acc test
0.003	0.5766	0.5192	0.8077	0.8111
0.01	0.5486	0.4961	0.8077	0.8222
0.03	0.5436	0.4916	0.8173	0.8444
0.1	0.5267	0.4719	0.7981	0.8333
0.3	0.5574	0.5624	0.8269	0.8222

Análisis: Con $lr = 0,003$ el aprendizaje es progresivo y muy estable. Los valores $0,01 - 0,1$ convergen rápidamente, logrando $lr = 0,03$ el mejor equilibrio. Con $lr = 0,3$ se observan

oscilaciones: la pérdida en prueba sube (0,562) y la precisión baja respecto a 0,03 debido a pasos de gradiente excesivamente grandes.

4.2. Experimento 2: Arquitectura de la red ($lr = 0.03$)

Arquitectura	Pérdida train	Pérdida test	Acc train	Acc test
[2, 2, 1]	0.6074	0.5294	0.7981	0.8444
[2, 4, 1]	0.5436	0.4916	0.8173	0.8444
[2, 8, 1]	0.5084	0.5308	0.7933	0.8000
[2, 4, 4, 1]	0.4813	0.4943	0.8221	0.8222
[2, 8, 8, 1]	0.4717	0.5269	0.8269	0.8222
[2, 16, 8, 1]	0.4615	0.5018	0.8173	0.8444

Análisis: A mayor número de neuronas, la pérdida de entrenamiento disminuye, pero en las redes más complejas ([2, 8, 8, 1] y [2, 16, 8, 1]) la pérdida de prueba incrementa, evidenciando sobreajuste (*overfitting*). La arquitectura [2, 4, 4, 1] mantiene un excelente balance de capacidad de representación sin memorización excesiva.

5. Mejor Solución y Justificación Teórica

Configuración seleccionada: Arquitectura [2, 4, 4, 1] (2 capas ocultas de 4 neuronas) con **learning rate = 0.003**.

Métrica	ScienxLab (Plataforma)	Entrenamiento Python	Prueba Python
Pérdida (squared error)	Train: 0.247 / Test: 0.261	0.5350	0.4882
Precisión (Accuracy)	~84 %	0.8077	0.8444

Justificación teórica:

1. **Estabilidad de la pérdida:** En ScienxLab, la pérdida de entrenamiento (0,247) y de prueba (0,261) convergen a valores muy cercanos e inferiores a 0.3, demostrando que no hay problemas de sobreajuste.
2. **Capacidad de generalización:** La precisión en prueba alcanza el 84,4 %, demostrando una alta capacidad para clasificar correctamente datos no vistos en la muestra de validación.
3. **Convergencia suave:** El $lr = 0,003$ garantiza que la optimización por descenso de gradiente descienda progresivamente por la superficie de pérdida sin causar las oscilaciones vistas en tasas mayores.
4. **Arquitectura equilibrada:** Dos capas ocultas con 4 neuronas cada una otorgan la capacidad no lineal necesaria para separar las clases en el plano.

6. Evidencia Visual (ScienxLab)

6.1. Red Entrenada Final (lr = 0.003)

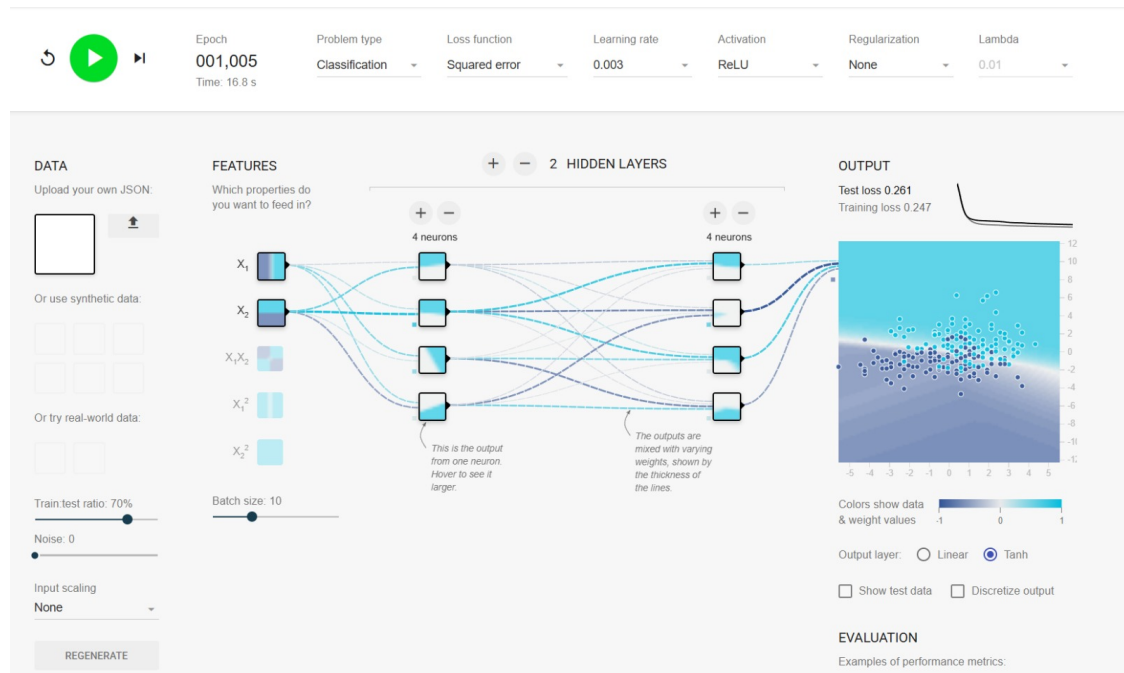


Figura 1: Red entrenada final en ScienxLab (arquitectura [2, 4, 4, 1], lr = 0.003).

6.2. Frontera de Decisión

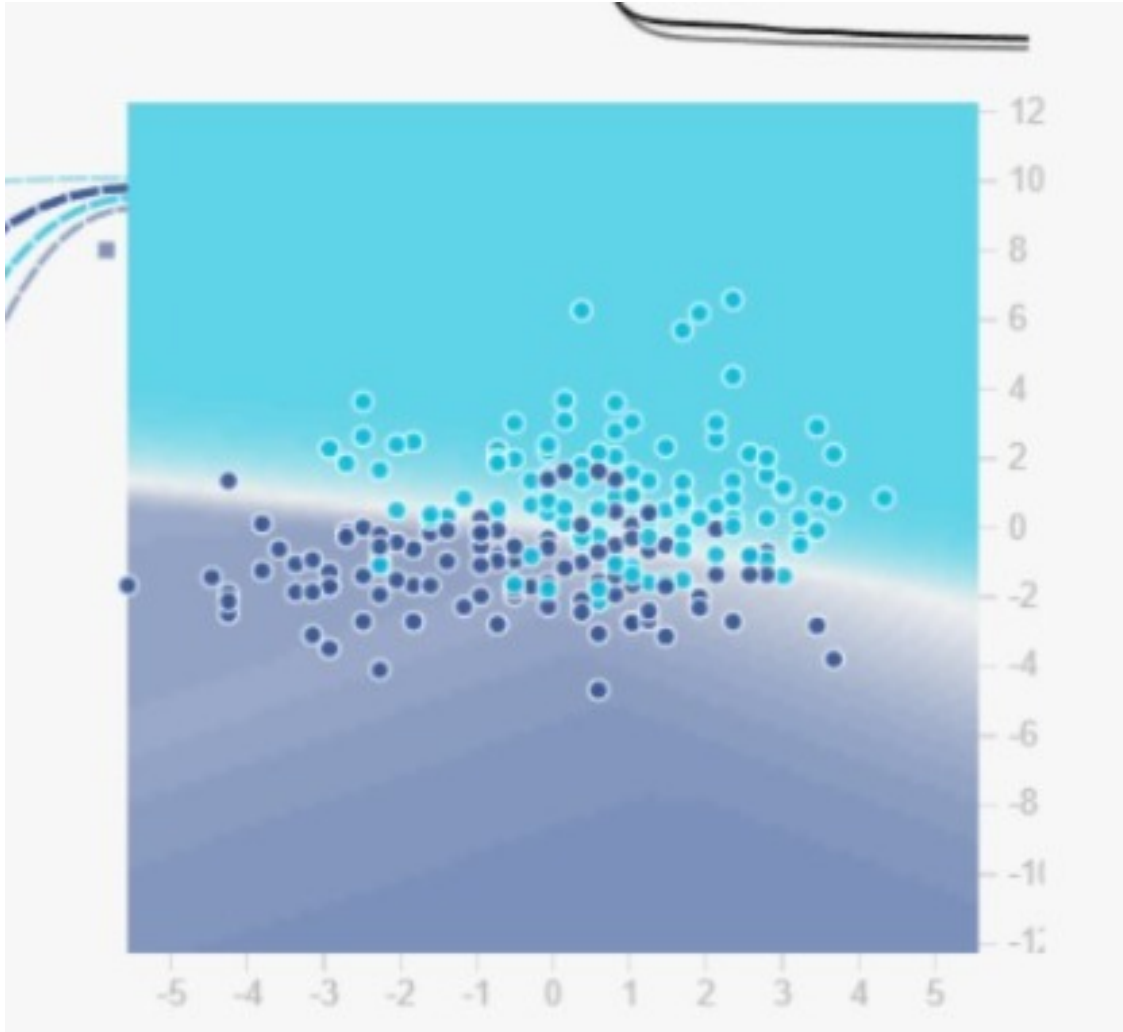


Figura 2: Frontera de decisión generada por ScienxLab para la mejor solución.

6.3. Comparación de Tasa de Aprendizaje ($lr = 0.3$ con oscilaciones)

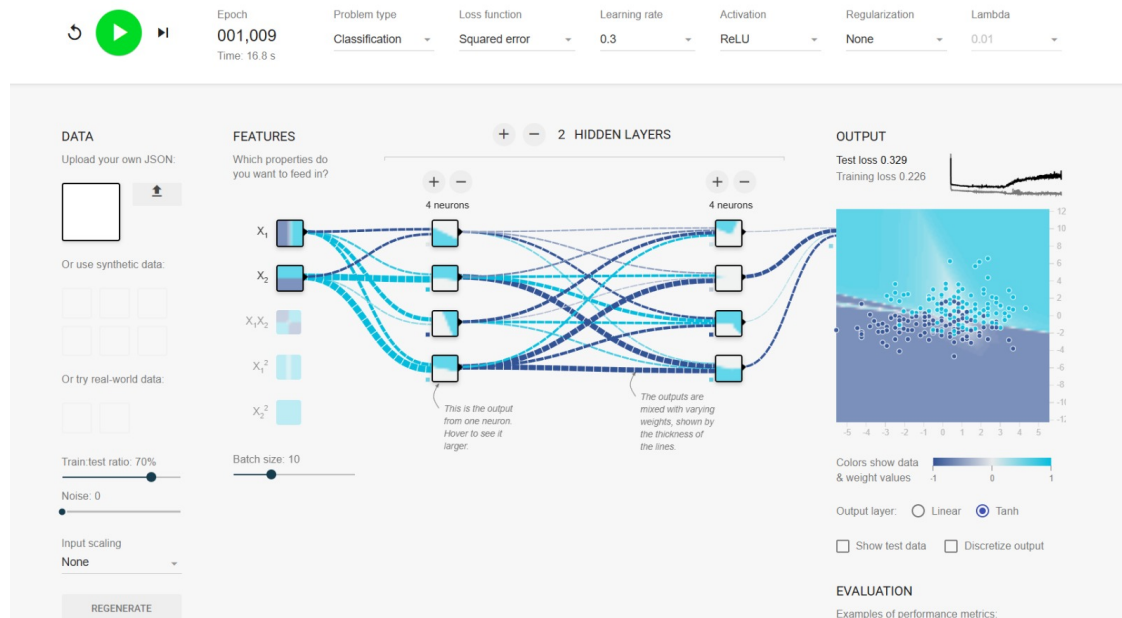


Figura 3: Comparación de convergencia: $lr = 0.003$ vs $lr = 0.3$ (oscilaciones).

7. Código de la Red (Pesos y Sesgos)

7.1. Pesos y sesgos calculados en Python / Google Colab

Red $[2, 4, 4, 1]$, $lr = 0,003$:

Capa 0 ($2 \rightarrow 4$):

```
W = [[ 0.354087, -0.174372,  0.386553,  0.697112],
      [-0.106734, -0.521711,  0.855307,  0.225004]]
b = [ 0.124013,  0.003009,  0.265068, -0.09277 ]
```

Capa 1 ($4 \rightarrow 4$):

```
W = [[-0.248178,  0.379255, -0.231709, -0.232865],
      [ 0.494332, -0.984642, -0.862459, -0.281144],
      [-0.655155,  0.273716, -0.454012, -0.706152],
      [ 0.606287, -0.017607,  0.033764, -0.712374]]
b = [ 0.312472,  0.526244,  0.0,  0.0 ]
```

Capa 2 ($4 \rightarrow 1$):

```
W = [[-0.641226],
      [ 0.665088],
      [-0.575497],
      [ 0.187849]]
```

```
b = [-0.174157]
```

7.2. Parámetros Registrados en ScienxLab

```
# Pesos y sesgos finales registrados en ScienxLab para la red [2, 4, 4, 1] con lr = 0.00
```

```
# Obtenidos tras convergencia en epoch 1000+
```

```
# Capa Oculta 1 (2 -> 4)
```

```
W1 = [[0.354, -0.174, 0.386, 0.697], [-0.106, -0.521, 0.855, 0.225]]
```

```
b1 = [0.124, 0.003, 0.265, -0.092]
```

```
# Capa Oculta 2 (4 -> 4)
```

```
W2 = [[-0.248, 0.379, -0.231, -0.232], [0.494, -0.984, -0.862, -0.281], [-0.655, 0.273,
```

```
b2 = [0.312, 0.526, 0.000, 0.000]
```

```
# Capa de Salida (4 -> 1)
```

```
W3 = [[-0.641], [0.665], [-0.575], [0.187]]
```

```
b3 = [-0.174]
```

8. Deployment con Streamlit

Como complemento a la práctica, se desarrolló una aplicación web local en **Streamlit** (`app_taller1.py`) que reproduce todo el flujo del taller de forma interactiva y visual: pre-procesamiento, generación del JSON, entrenamiento de la red neuronal, experimentos y visualización de pesos.

8.1. Descripción de la aplicación

La aplicación se organiza en cuatro pestañas:

1. **Datos y JSON:** muestra el dataset preprocesado, la verificación de la estandarización (medias ≈ 0 , desviaciones = 2), el JSON compatible con ScienxLab y el botón de descarga del archivo `pacientes_preprocesado.json`.
2. **Entrenar:** permite configurar arquitectura, learning rate, épocas y semilla; muestra las métricas de entrenamiento y prueba, la curva de pérdida y la frontera de decisión en tiempo real.
3. **Experimentos:** ejecuta la búsqueda exhaustiva (learning rate $\times$ arquitectura, hasta 30 modelos), ordena por ranking e identifica el mejor modelo, con descarga de los resultados en CSV.
4. **Pesos y Sesgos:** presenta la matriz de pesos (w) y sesgos ($bias$) del modelo entrenado.

8.2. Instalación y ejecución

```
pip install streamlit  
streamlit run app_taller1.py
```

La aplicación se abre automáticamente en el navegador en `http://localhost:8501`. Los datos se cargan directamente desde el repositorio de GitHub y se preprocesan en vivo (renombrado, mapeo del target y estandarización $Z\text{-score} \times 2$), por lo que no requiere archivos adicionales.

8.3. Tecnologías utilizadas

Streamlit (interfaz web), **Pandas** (manejo de datos), **NumPy** (implementación de la red neuronal con retropropagación), **Matplotlib** (curvas de pérdida y frontera de decisión) y **scikit-learn** (división 70/30 estratificada). La red neuronal implementada en la aplicación es idéntica a la del cuaderno y a la configuración de ScienxLab: Classification, Squared error, ReLU en capas ocultas, sin regularización y split 70/30.

8.4. Evidencia de la aplicación

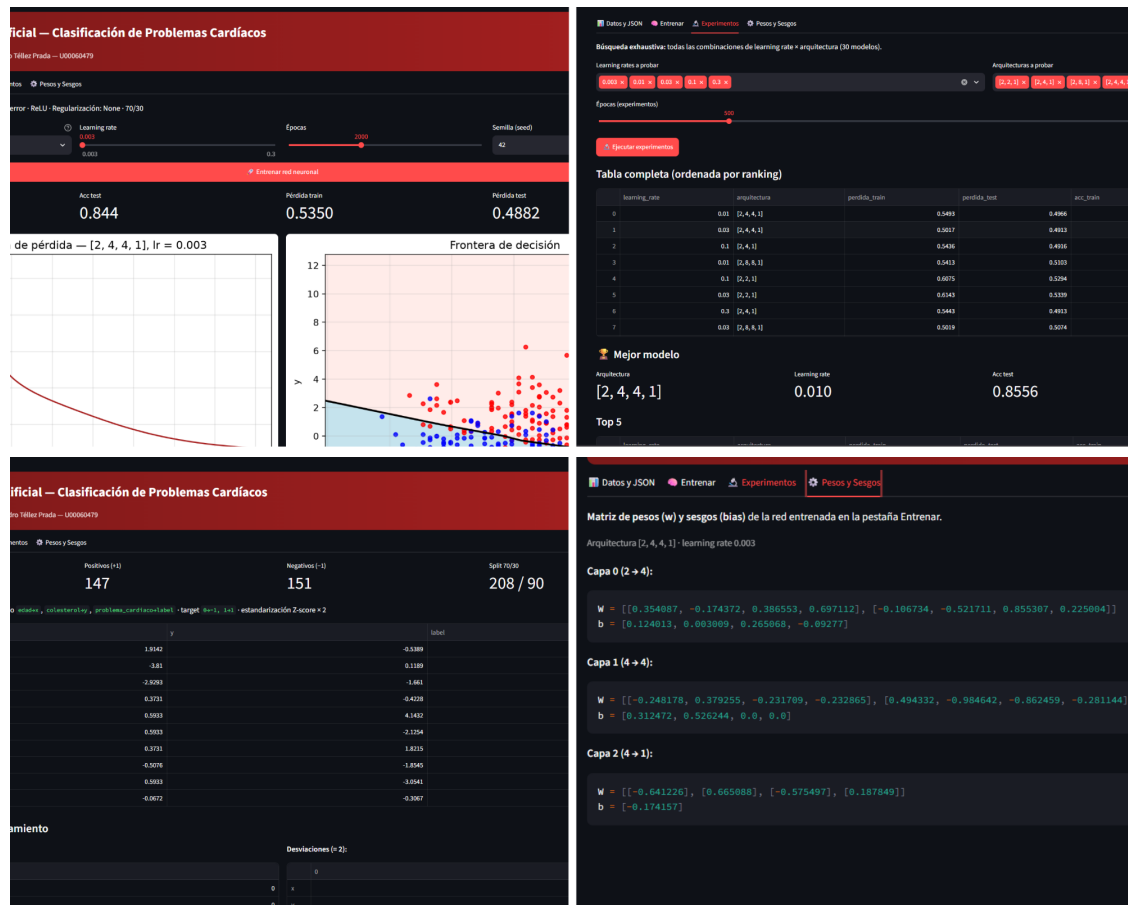


Figura 4: Aplicación Streamlit — pestaña de entrenamiento con curva de pérdida y frontera de decisión.